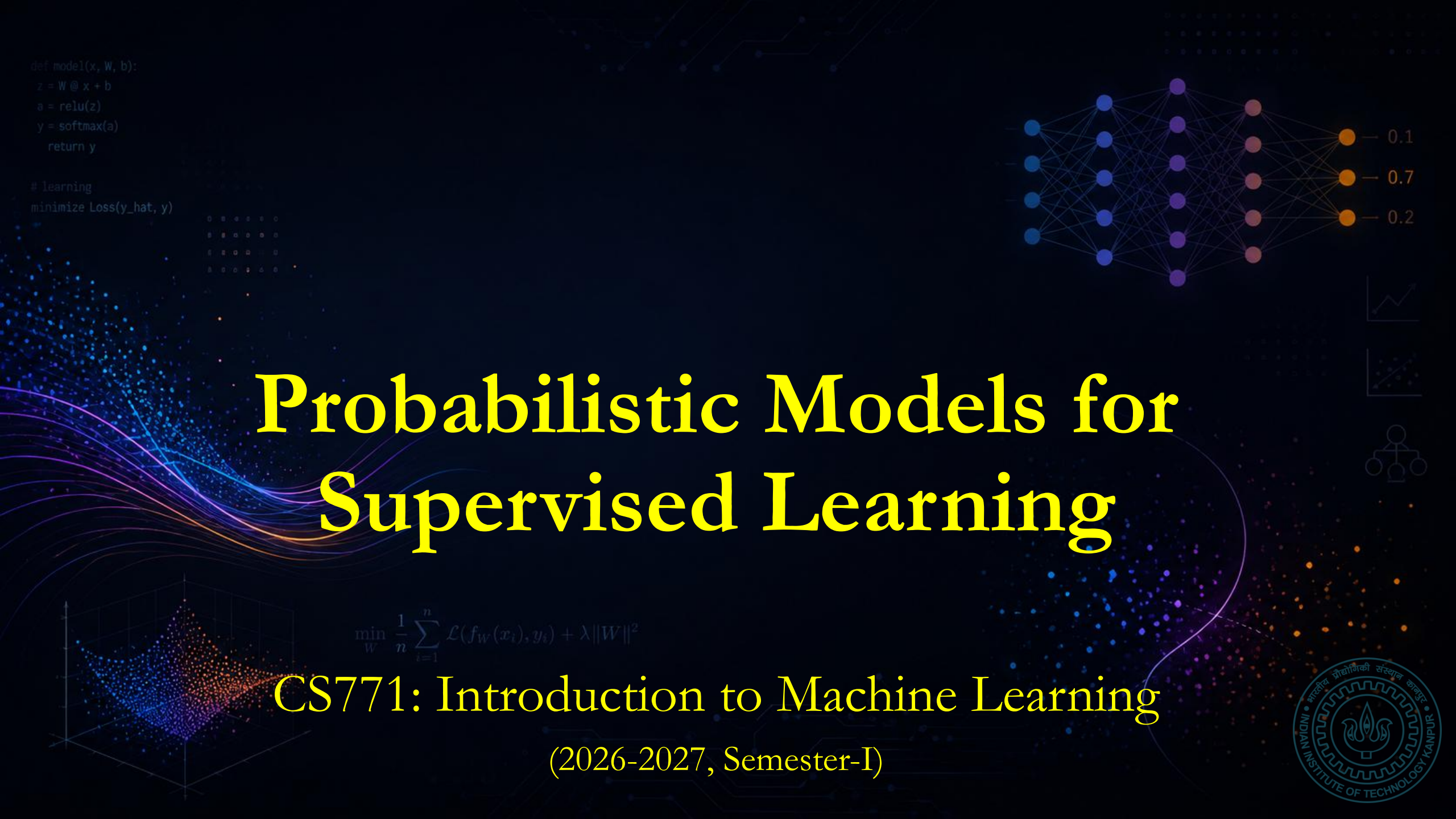


Probabilistic Models for Supervised Learning

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

Probabilistic Supervised Learning

Earlier we modeled y_n using a distribution $p(y_n|\theta)$

For example, $\text{Bernoulli}(y_n|\theta)$ $\mathcal{N}(y_n|\mu, \sigma^2)$

Often, for each y_n we also have an associated $\boldsymbol{x}_n$

- In supervised learning, y_n is output and $\boldsymbol{x}_n$ is input features

In such cases, we may assume the distribution of y_n to take the form

$$p(y_n|\theta_n) = p(y_n|f(\boldsymbol{x}_n))$$

Now a different parameter θ_n for the distribution of each y_n

The parameter θ_n is now some function f of the input $\boldsymbol{x}_n$

Probabilistic model for y_n will now be the conditional distribution $p(y_n|\boldsymbol{x}_n, \boldsymbol{w})$ and parameter estimation would boil down to estimating $\boldsymbol{w}$ that defines this conditional distribution

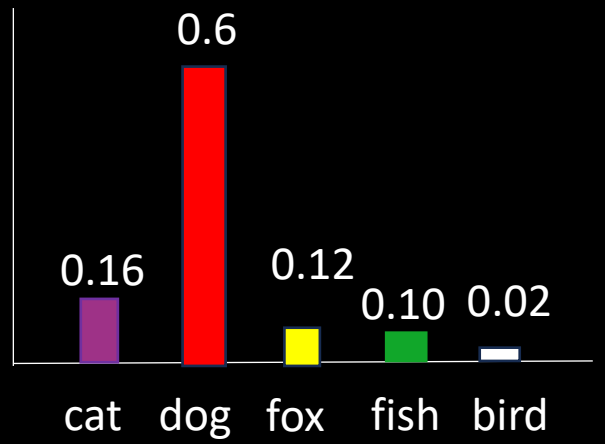
The function f may itself depend on parameters $\boldsymbol{w}$, e.g., $f(\boldsymbol{x}_n) = \boldsymbol{w}^\top \boldsymbol{x}_n$ or $\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n)$

What does $p(y|x)$ mean in Supervised Learning?

Classification

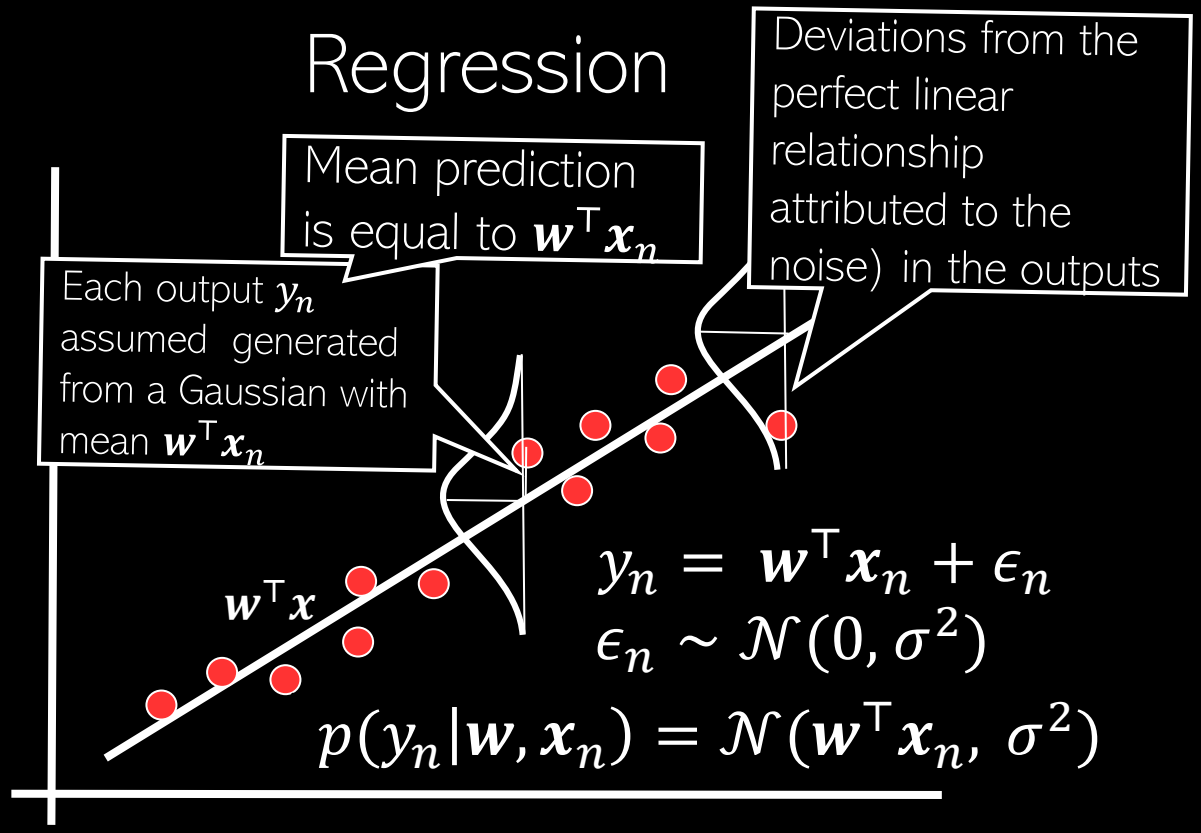
$$p(y_n|\boldsymbol{w}, \boldsymbol{x}_n) = \text{multinoulli}(\text{softmax}(\boldsymbol{W}^\top \boldsymbol{x}_n))$$

$\boldsymbol{x}_n =$ 



$p(y_n = c \mid \boldsymbol{x}_n, \boldsymbol{w})$: probability that the label $y_n = c$

Regression



$p(y_n = t|\boldsymbol{x}_n, \boldsymbol{w})$: probability that $y_n \in (t, t + \delta t)$.

Probabilistic Binary Classification (Logistic Reg.)

Earlier we modeled $y_n \in \{0,1\}$ using $p(y_n) = \text{Bernoulli}(y_n|\theta)$

Assume y also depends on some inputs (say "features") $\boldsymbol{x}_n \in \mathbb{R}^D$

$$p(y_n|\boldsymbol{x}_n) = \text{Bernoulli}(y_n|\mu_n) = \text{Bernoulli}(y_n|\sigma(\boldsymbol{w}^\top \boldsymbol{x}_n))$$

Now a different μ for each example

The bias term b of $w^\top x + b$ assumed to be subsumed in $\boldsymbol{w}$

Goal: estimate $\boldsymbol{w}$. Learning principles for MLE, MAP, posterior, predictive, remain similar to the $p(y_n) = \text{Bernoulli}(y_n|\theta)$ case

Given training data $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^N$ NLL (exercise)

The logistic or binary cross-entropy loss

Bernoulli likelihood for binary classification = logistic/cross-entropy loss

$$\text{NLL}(\boldsymbol{w}) = \sum_{n=1}^{N} \left[\log\left(1 + \exp(\boldsymbol{w}^\top \boldsymbol{x}_n)\right) - y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right]$$

Minimizing NLL will give us the MLE solution for $\boldsymbol{w}$

Note: If $y \in \{-1, +1\}$, then $\text{NLL} = \sum_{n=1}^{N} \left[\log\left(1 + \exp(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n)\right)\right]$



Probabilistic Multiclass Classifn (Softmax Reg.)

For multi-class case, $y_n \in \{1,2,\ldots,C\}$ we can use

$$\mu_{n,j} = \frac{\exp(\boldsymbol{w}_j^\top \boldsymbol{x}_n)}{\sum_{k=1}^{C} \exp(\boldsymbol{w}_k^\top \boldsymbol{x}_n)}$$

Probability that input $\boldsymbol{x}_n$ belongs to class j

$$p(y_n|\boldsymbol{x}_n) = \text{multinoulli}(y_n|\boldsymbol{\mu}_n)$$

Softmax here returns a $C \times 1$ vector of class probabilities $\mu_n = [\mu_1, \mu_2, \ldots, \mu_C]$ for the input $\boldsymbol{x}_n$

$$= \text{multinoulli}(y_n|\text{softmax}(\boldsymbol{W}^\top \boldsymbol{x}_n))$$

$\boldsymbol{W}$ is the $D \times C$ matrix

Goal: estimate $\boldsymbol{W} = [\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots \boldsymbol{w}_C]$ where $\boldsymbol{w}_j \in \mathbb{R}^D, j = 1,2,\ldots,C$

NLL w.r.t. a single training example $(\boldsymbol{x}_n, y_n)$ can be written as

$$-\log p(y_n|\boldsymbol{x}_n) = -\log \prod_{j=1}^{C} \mu_{n,j}^{\mathbb{I}[y_n=j]} = -\sum_{j=1}^{C} \mathbb{I}[y_n = j] \log \mu_{n,j} = -\log \mu_{n,y_n}$$

Thus NLL is multi-class cross-entropy loss: Its argmin gives MLE for $\boldsymbol{W}$

Probabilistic Linear Regression

Earlier we modeled $y_n \in \mathbb{R}$ using $p(y_n) = \mathcal{N}(y_n|\mu, \sigma^2)$

Assume y_n depends on some inputs (say "features") $\boldsymbol{x}_n \in \mathbb{R}^D$

$$p(y_n|\boldsymbol{x}_n) = \mathcal{N}(y_n|\mu_n, \sigma^2) = \mathcal{N}(y_n|\boldsymbol{w}^\top \boldsymbol{x}_n, \sigma^2)$$

Now a different μ for each input $\boldsymbol{x}_n$

The bias term b of $w^\top x + b$ assumed to be subsumed in $\boldsymbol{w}$

Goal: estimate $\boldsymbol{w}$ (assume σ^2 known). Learning principles for MLE/MAP, posterior, predictive, remain similar to the $p(y_n) = \mathcal{N}(y|\mu, \sigma^2)$ case

Given training data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ NLL (exercise)

$$\text{NLL}(w) = \frac{1}{2\sigma^2} \sum_{n=1}^{N} (y_n - w^\top x_n)^2$$

Gaussian likelihood for regression = squared loss function

Minimizing NLL will give us the MLE solution for $\boldsymbol{w}$



Probabilistic Linear Regression: Some Variants

Laplace distribution to model the output distribution/likelihood

$$p(y_n|x_n) = \text{Lap}(y_n|\mu_n, b) \propto \exp\left[-\frac{1}{b}|y_n - \boldsymbol{w}^\top \boldsymbol{x}_n|\right]$$

Mean $\boldsymbol{w}^\top \boldsymbol{x}_n$

Variance like term

This likelihood corresponds to the absolute loss function for regression

Gaussian distribution for the output but with heteroskedastic noise

$$p(y_n|x_n) = \mathcal{N}(y|\mu_n, \sigma_n^2)$$

Different variance for the Gaussian noise $\epsilon_n \sim \mathcal{N}(0, \sigma_n^2)$ assumed in each $y_n = w^\top x_n + \epsilon_n$

$$= \mathcal{N}(y|\boldsymbol{w}^\top \boldsymbol{x}_n, \sigma_n^2)$$

The variance can even be input-dependent, e.g., $\sigma_n^2 = \exp(\boldsymbol{\gamma}^\top \boldsymbol{x}_n)$ and $\gamma \in \mathbb{R}^D$ has to be learned

Helps us compute input dependent predictive uncertainty (variance in the predicted output)

MLE/MAP usually straightforward for these variants

Prior over the Weight Vector $\boldsymbol{w}$

A $\boldsymbol{D}$-dim Gaussian prior is a natural choice for $\boldsymbol{w} \in \mathbb{R}^D$

In general, it can be

$$p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\boldsymbol{w}_0, \boldsymbol{\Sigma}_0)$$

a priori we expect/believe the solution to be close to some fixed vector $\boldsymbol{w}_0$

$\boldsymbol{\Sigma}_0$ controls the variance of this prior belief

Very commonly, we use $\boldsymbol{w}_0 = \boldsymbol{0}$ and $\boldsymbol{\Sigma}_0 = \lambda^{-1}\boldsymbol{I}_D$

$$p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\boldsymbol{0}, \lambda^{-1}\boldsymbol{I}_D) = \left(\frac{\lambda}{2\pi}\right)^{D/2} \exp\left[-\frac{\lambda}{2}\boldsymbol{w}^\top \boldsymbol{w}\right]$$

Thus using this prior in MAP estimation is akin to using ℓ_2 regularizer on $\boldsymbol{w}$

The prior $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\boldsymbol{0}, \lambda^{-1}\boldsymbol{I}_D)$ encourages weights to be small

The precision hyperparam $\boldsymbol{\lambda}$ acts like the regularization hyperparameter

MAP Estimate and Posterior Distribution of $\boldsymbol{w}$

MAP estimation is straightforward for any likelihood

$$\boldsymbol{w}_{MAP} = \arg\min_{w} \left[\mathrm{NLL}(\boldsymbol{w}) + \frac{\lambda}{2} \boldsymbol{w}^\top \boldsymbol{w} \right]$$

ℓ_2 regularized logistic regression or linear regression

Posterior is also easy for linear regression

Thanks to conjugacy! Likelihood is Gaussian, prior is Gaussian; Gaussian is conjugate to itself

$$p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y}) = \frac{p(\boldsymbol{w})p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w})}{p(\boldsymbol{y}|\boldsymbol{X})}$$

$$= \mathcal{N}(\boldsymbol{w}|\boldsymbol{\mu}_N, \boldsymbol{\Sigma}_N)$$

Posterior has the same form – Gaussian - as the prior, and the exact posterior can be easily calculated analytically

where

$$\boldsymbol{\mu}_N = (\boldsymbol{X}^\top \boldsymbol{X} + \frac{\lambda}{\beta} \boldsymbol{I}_D)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

$$\boldsymbol{\Sigma}_N = (\beta \boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I}_D)^{-1}$$

Note: In the Gaussian posterior case, the posterior's mean is the same as the MAP solution

For logistic/softmax regression, the likelihood (Bernoulli/multinoulli) is NOT conjugate to the prior (Gaussian) on $\boldsymbol{w}$. Can't compute posterior in closed form



Posterior, when the likelihood is not conjugate to the Gaussian prior on $\boldsymbol{w}$, has to be approximated (methods exist, such as Laplace's approximation, MCMC, variational inference (beyond the scope of this course))

What does posterior over $\boldsymbol{w}$ mean?

Probability density $p(\boldsymbol{w}|X, y)$ tells us how well a specific $\boldsymbol{w}$ is supported by data and prior

Not all are equally good, of course!

Posterior distribution $p(\boldsymbol{w}|X, y)$ represents a continuum of solutions

Can visualize by randomly "sampling" these solutions from $p(\boldsymbol{w}|X, y)$

Regression example: Assuming bias w_0 subsumed in $\boldsymbol{w} = [w_1, w_0]$, a random draw will give us a sample

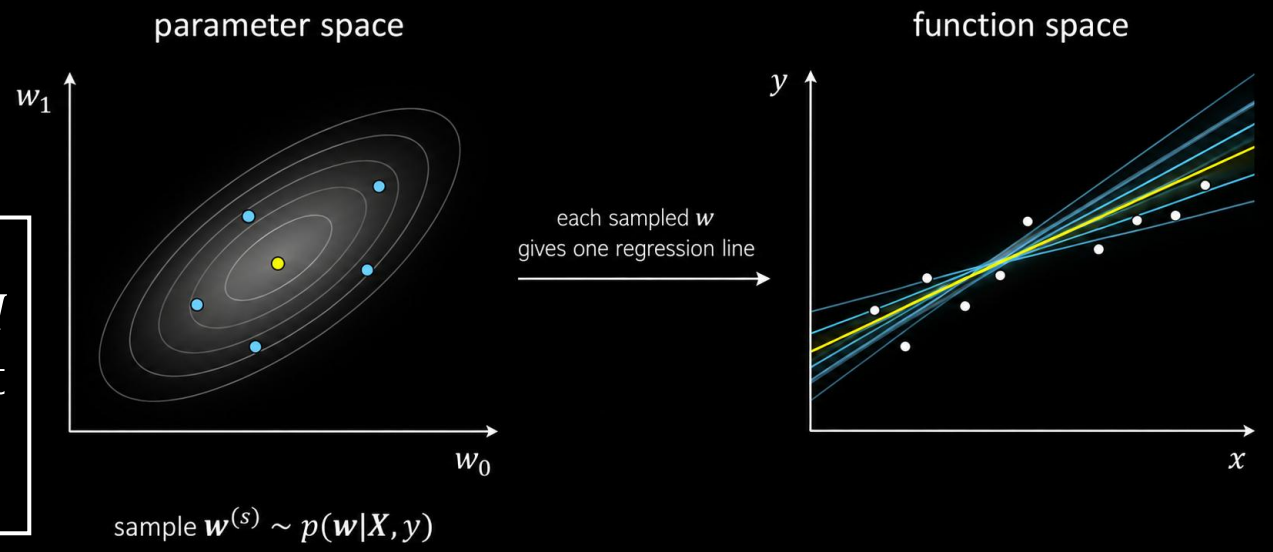
$$\boldsymbol{w}^{(s)} = [w_1^{(s)}, w_0^{(s)}]$$

Regression line with slope $w_1^{(s)}$ and intercept $w_0^{(s)}$

The posterior defines a continuum of such lines (a finite of them shown here)

Using M such randomly drawn lines, we can get M predictions for y_* of a test input x_* and report the mean and variance of y_*

Samples from posterior $p(\boldsymbol{w}|X, y)$



What does posterior over $\boldsymbol{w}$ mean?

Posterior distribution $p(\boldsymbol{w}|X, y)$ represents a continuum of solutions

Can visualize by randomly "sampling" these solutions from $p(\boldsymbol{w}|X, y)$

Classification (logistic regression) example: Assuming hyperplane separator in $\boldsymbol{w} = [w_1, w_2]$, a random draw will give us a sample

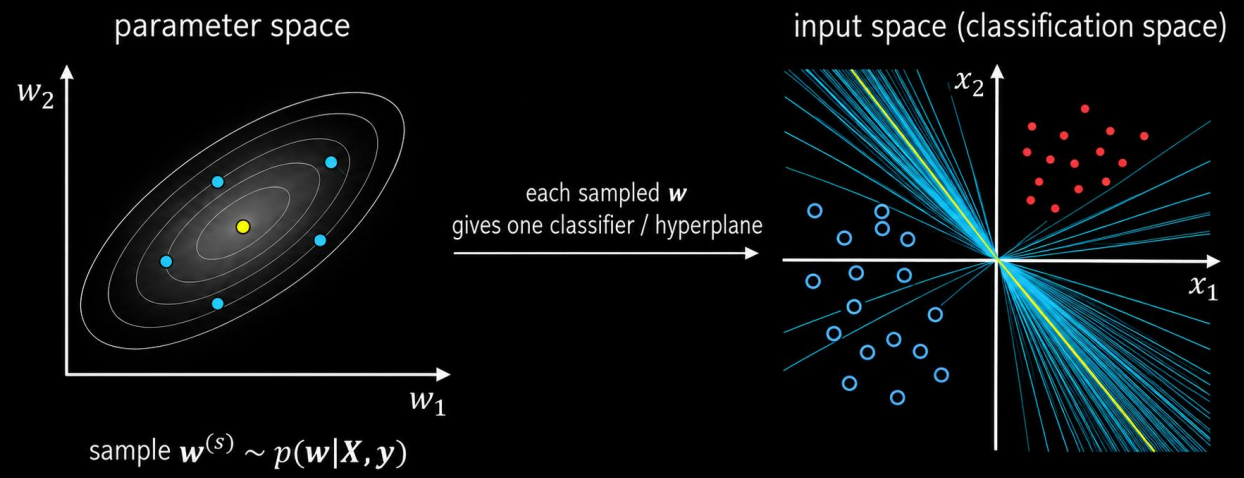
$$w^{(s)} = [w_1^{(s)}, w_2^{(s)}]$$

The posterior defines a continuum of such hyperplanes

A finite many of them shown here

Using M such randomly drawn hyperplanes, we can get M values for $p(y_* = 1|x_*)$ and report their average and variance

Samples from posterior $p(\boldsymbol{w}|X, y)$



Making Predictions

Note: For probabilistic linear regression with Gaussian lik and Gaussian prior, the predictive distribution can be analytically calculated if only $\boldsymbol{w}$ is unknown (and other params/hyperparams fixed)

$$p(y_*|\boldsymbol{x}_*, \boldsymbol{X}, \boldsymbol{y}) = \mathcal{N}(y_*|\mu_N^\top x_*, \sigma^2 + x_*^\top \Sigma_N x_*)$$



The predictive distribution will be

Conditioned on the training data $(\boldsymbol{X}, \boldsymbol{y})$ and test input $\boldsymbol{x}_*$

$$p(y_*|\boldsymbol{x}_*, \boldsymbol{X}, \boldsymbol{y}) = \int p(y_*|\boldsymbol{w}, \boldsymbol{x}_*) p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y}) d\boldsymbol{w}$$

If the integral is not tractable then the above can be approximated using randomly drawn samples from the posterior

$$w^{(1)}, w^{(2)}, \ldots, w^{(M)} \overset{\text{iid}}{\sim} p(w \mid \boldsymbol{X}, \boldsymbol{y})$$

$$p(y_*|\boldsymbol{x}_*, \boldsymbol{X}, \boldsymbol{y}) \approx \frac{1}{M} \sum_{m=1}^{M} p\left(y_* \mid w^{(m)}, x_*\right)$$

Example: In the logistic regression case, $p(y_* = 1|\boldsymbol{x}_*, \boldsymbol{X}, \boldsymbol{y}) = \frac{1}{M} \sum_{m=1}^{M} \sigma(\boldsymbol{w}_m^\top \boldsymbol{x}_*)$

Average of $\boldsymbol{M}$ sigmoids!

But if using MLE/MAP estimate $\widehat{\boldsymbol{w}}$, the predictive distribution will be

$$p(y_*|\boldsymbol{x}_*, \boldsymbol{X}, \boldsymbol{y}) \approx p(y_*|\widehat{w}, \boldsymbol{x}_*)$$

Example: In the logistic regression case, $p(y_* = 1|\boldsymbol{x}_*, \boldsymbol{X}, \boldsymbol{y}) = \sigma(\widehat{w}^\top \boldsymbol{x}_*)$

Probabilistic Modeling Beyond Linear Models

All of these ideas also hold if we replace $\boldsymbol{x}$ by $\boldsymbol{\phi}(\boldsymbol{x})$ where $\boldsymbol{\phi}$ denotes an appropriate nonlinear feature mapping

Or more generally, if we replace $\boldsymbol{w}^\top \boldsymbol{x}$ by $\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{W})$ where $\boldsymbol{f}$ denotes a deep neural network with weights $\boldsymbol{W}$

For example:

$$p(y|\boldsymbol{x}) = \mathcal{N}(y|\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}), \sigma^2)$$
$$p(y|\boldsymbol{x}) = \mathcal{N}(y|\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{W}), \sigma^2)$$
$$p(y|\boldsymbol{x}) = \text{multinoulli}(y|\text{softmax}(W^\top \phi(x)))$$
$$p(y|\boldsymbol{x}) = \text{multinoulli}(y|\text{softmax}(\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{W})))$$

With such likelihoods, we can minimize the NLL to do MLE

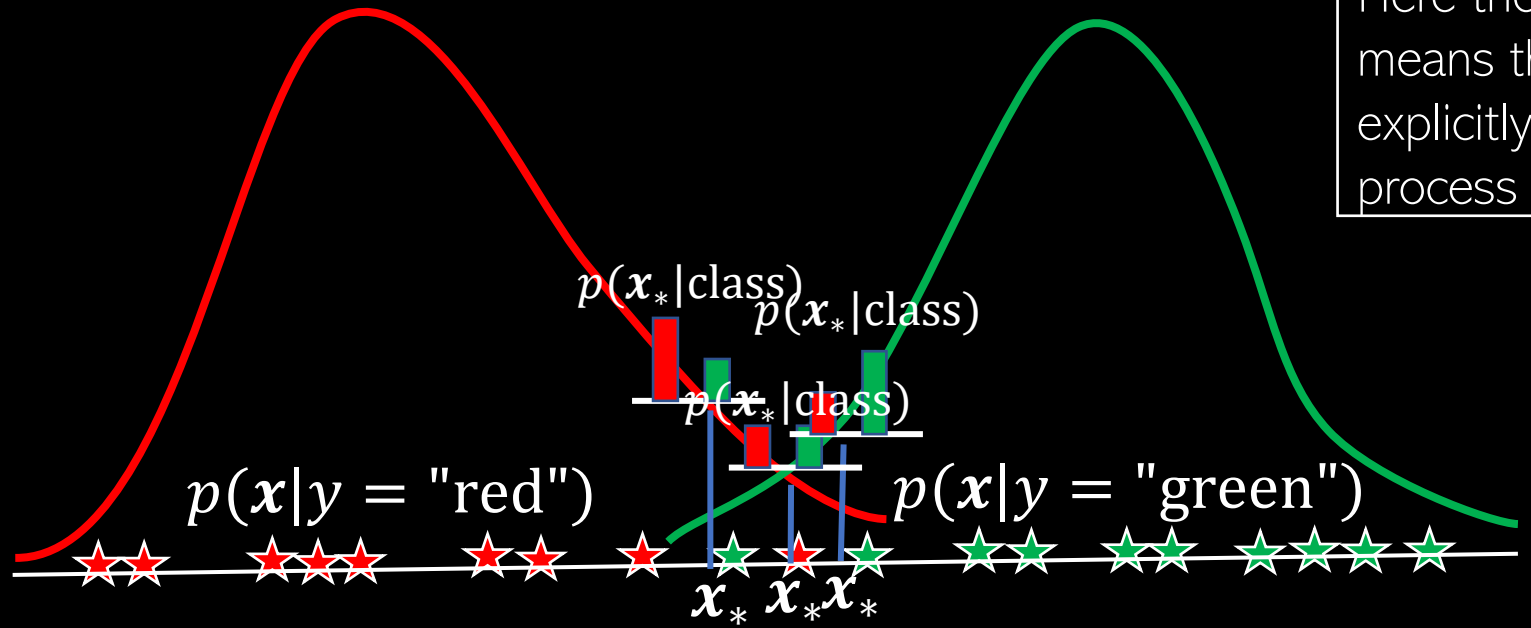
Can also use appropriate priors on the weights to do MAP estimation or even compute the posterior (usually exact computation is not possible but can approximate)



A Generative Approach to Classification

Assume each input $\boldsymbol{x}$ of class $\boldsymbol{c}$ is generated from $p(\boldsymbol{x}|y = c)$

Estimate the distribution of inputs from each class using training data



Here the word "generative" means that the method explicitly defines the generation process of the inputs $\boldsymbol{x}$

This simple idea is the basis of "generative classification" methods



Note: we can actually do better as we will see shortly, but this is the basic idea

We can now compare $p(\boldsymbol{x}_*|y_* = c)\ \forall c$ to predict label of test input $\boldsymbol{x}_*$

Generative Classification

Conditional probability that the label $y_n = c$ given the input $\boldsymbol{x}_n$

$$p(y_n = c|x_n) = \frac{p(\boldsymbol{x}_n, y_n = c)}{p(x_n)}$$

$$= \frac{p(y_n = c)p(\boldsymbol{x}_n|y_n = c)}{\sum_{i=1}^{C} p(y_n = i)p(\boldsymbol{x}_n|y_n = i)}$$

Assumed probability distribution of the inputs from class c

Known as the "class-conditional" distribution of the inputs from class c

$p(y_n = c)$ is the "overall" probability of class c

- *Roughly*, it tells us what fraction of inputs belong to class c
- Also called the "class marginal" or "class prior"

But only if estimated using MLE; otherwise, the estimate may be slightly different from the fraction

Not to be confused with a prior distribution over parameters

Estimating Overall Class Probability $p(y)$

In classification, labels y are discrete (binary or multiclass/categorical)

For the binary case, $p(y = 1) = \pi$

$$p(y|\pi) = \text{Bernoulli}(y|\pi) = \pi^y (1-\pi)^{1-y}$$

For the multiclass case, $p(y = i) = \pi_i$

These probabilities must sum to 1: $\sum_{i=1}^{C} \pi_i = 1$

$$p(y|\boldsymbol{\pi}) = \text{multinoulli}(y|\pi_1, \pi_2, \ldots, \pi_C) = \prod_{i=1}^{C} \pi_i^{\mathbb{I}[y=i]}$$

To estimate $\boldsymbol{\pi}$, we only need labels $\{y_1, y_2, \ldots, y_N\}$ from training data

MLE solution for $\boldsymbol{\pi}$ = fraction of labels from respective classes (easy!)

Binary case

$$\hat{\pi} = \frac{N_1}{N}$$

Multiclass case

$$\hat{\pi}_i = \frac{N_i}{N} \qquad i = \{1,2, \ldots, C\}$$

MLE is often adequate; but when training data is very little, a prior can stabilize the estimate by computing MAP or the full posterior.



Estimating Class Conditionals $p(x|y=c)$

We typically model $p(x|y=c)$ using some distribution $p(x|\theta_c)$

Can estimate parameters θ_c using inputs from class c

Can consider spherical or diagonal covariance instead of full covariance

Assuming $p(x|\theta_c) = \mathcal{N}(x|\mu_c, \Sigma_c)$ and if using MLE

Should be careful if input dimensionality is large

$$\hat{\mu}_c = \frac{1}{N_c}\sum_{n:y_n=c} x_n$$

$$\hat{\Sigma}_c = \frac{1}{N_c}\sum_{n:y_n=c} (x_n - \hat{\mu}_c)(x_n - \hat{\mu}_c)^\top$$

Gaussian class-conditional is one of the simplest choices

Other distributions can also be used depending on the nature of the inputs (even deep generative models can be used to estimate the distribution $p(x|\theta_c)$ making generative classification very powerful)

Generative Classification: Prediction



$\hat{\pi}_c$ takes into account how common class c is in the training data

The distribution $p(x_*|\hat{\theta}_c)$ explicitly takes into account the shape of class c

To predict label for test input $\boldsymbol{x}_*$ we compute

$$p(y_* = c|\boldsymbol{x}_*) = \frac{p(y_* = c)p(\boldsymbol{x}_*|y_* = c)}{p(\boldsymbol{x}_*)} \propto p(y_* = c) \times p(\boldsymbol{x}_*|y_* = c) \; \forall c$$

$$\boxed{\hat{y}_* = \arg\max_c \; p(y_* = c|\boldsymbol{x}_*)}$$

Assuming we have point estimates for parameters, then

$$\boxed{p(y_* = c|\boldsymbol{x}_*) \propto \hat{\pi}_c p(x_*|\hat{\theta}_c)} \; \forall c$$

If overall class probabilities are equal (i.e., $\hat{\pi}_c = 1/C$) then

$$\boxed{p(y_* = c|\boldsymbol{x}_*) \propto p(x_*|\hat{\theta}_c)} \; \forall c$$

$\boldsymbol{\Sigma}_c$ used for each class also dictates the shape of decision boundaries between classes (e.g., linear, quadratic)

Try as an exercise

Same $\boldsymbol{\Sigma}_c = \sigma^2\mathbf{I}$ for all classes, Using other forms of $\boldsymbol{\Sigma}_c$ will give us variants of LwP that don't use Euclidean but other distances, such as Mahalanobis distance

Using $\hat{\pi}_c = 1/C$, $p(\boldsymbol{x}|y = c) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_c, \sigma^2\mathbf{I})$ gives LwP with Eucl. dist.

Generative Supervised Learning: Some Comments

We already saw that it allows taking into account class popularities (via $\hat{\pi}_c$) and the shape of each class (via $p(\boldsymbol{x}|\hat{\theta}_c)$)

It also enables proper handling of

- Missing/corrupted features
- Missing/corrupted labels

What "proper" means will become clear when we discuss latent variable models

Though commonly used for classification, can be used for regression too

Continuous y

Use a suitable density

$$p(y|x) = \frac{p(x,y)}{p(x)} = \frac{p(y)p(x|y)}{p(x)}$$

Diff from classification: Since y is continuous, we won't have just finite-many $p(x|y=c)$ "class conditionals"

Solution 1: Define $p(x|y)$ such that its parameters are functions of continuous y

Solution 2: Estimate the joint distribution $p(x,y)$ directly

Probabilistic Supervised Learning: The Two Ways

We have seen two ways to compute $p(y|x)$ in supervised learning

(1) Model $p(y|x)$ directly as a distribution and treat inputs as "given" but not model them explicitly, e.g.,

$$p(y|x) = \text{Bernoulli}(y|\sigma(w^\top x))$$
$$p(y|x) = \text{multinoulli}(y|\text{softmax}(W^\top x))$$
$$p(y|x) = \mathcal{N}(y|w^\top x, \sigma^2)$$

Here we don't make any effort in modeling inputs x, and simply treat them as "given" and use them directly to condition the distribution $p(y|x)$

(2) Model $p(y|x)$ by modeling $p(y)$ and $p(x|y)$ and using

$$p(y|x) = \frac{p(y)p(x|y)}{p(x)}$$

Both have their pros and cons depending on problem setting, data availability, and the questions asked

(1) is called discriminative supervised learning; (2) is called generative supervised learning. Long-standing debate over which approach is better